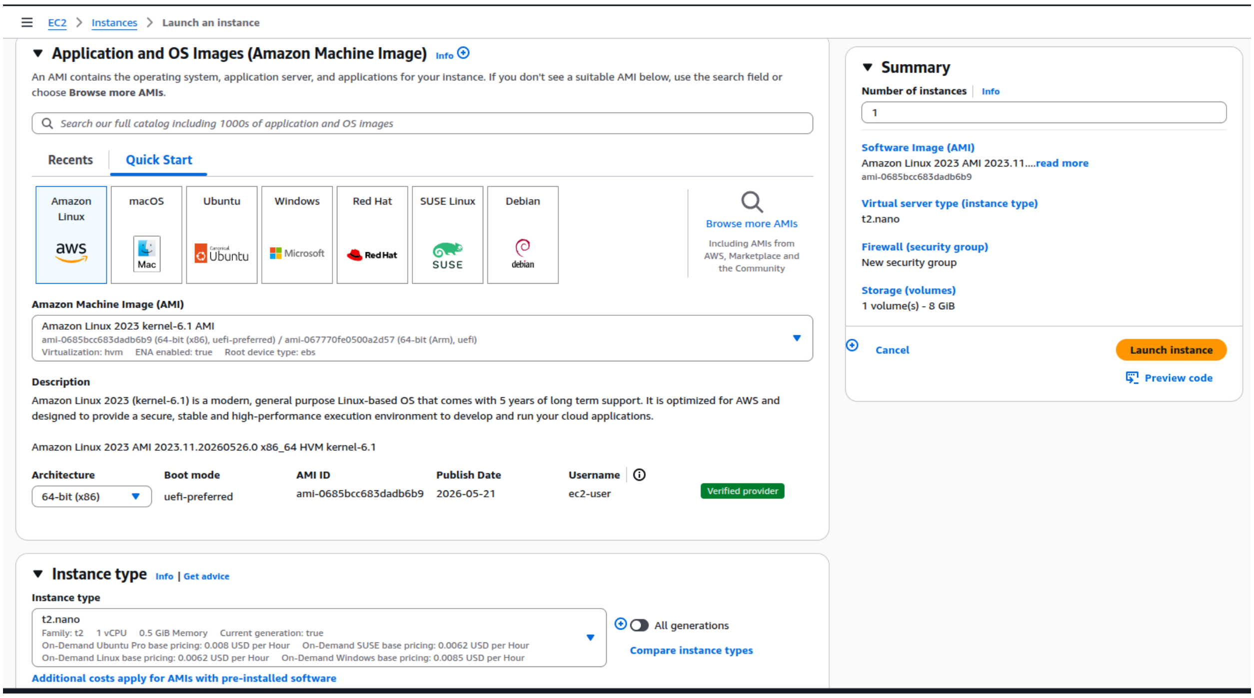
Simple AWS Auto Scaling Project

Phase 1

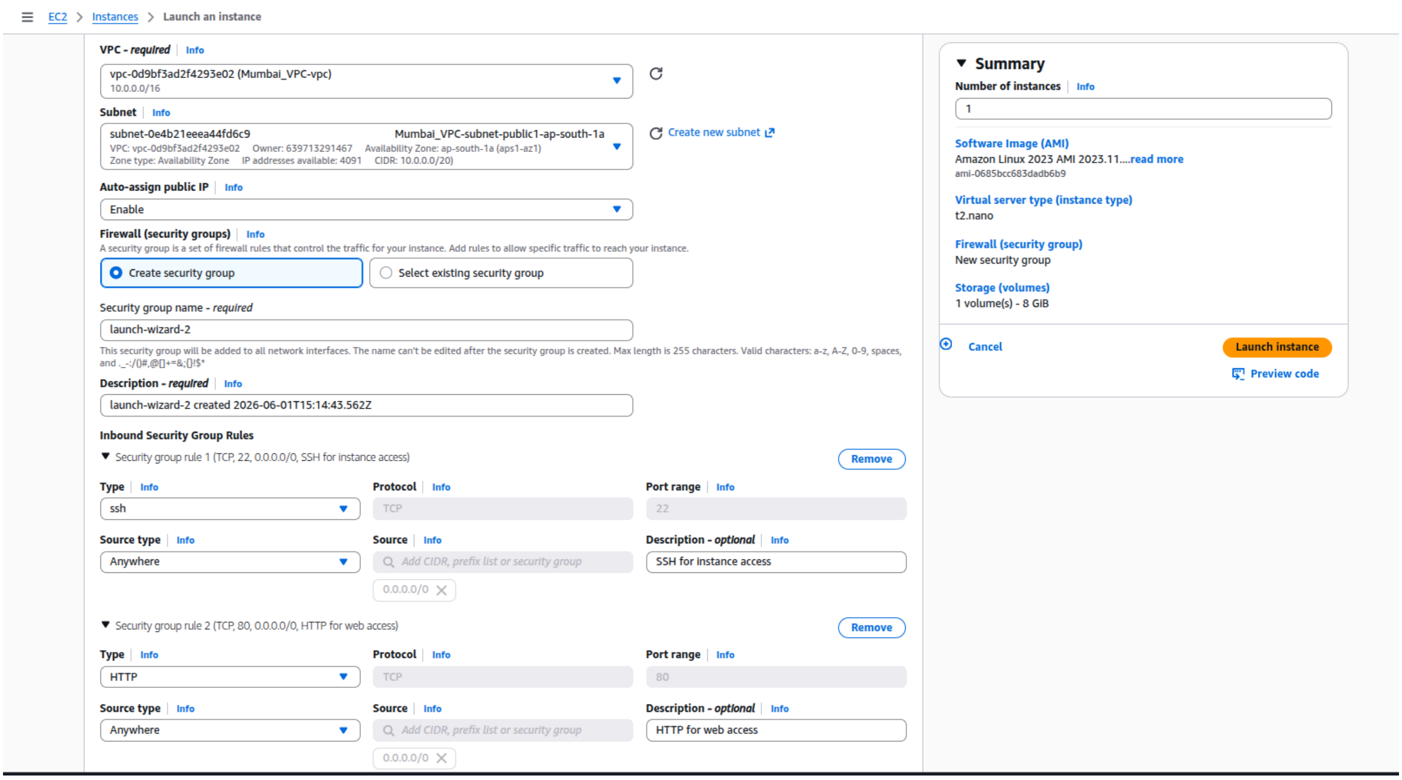
Create EC2 instance

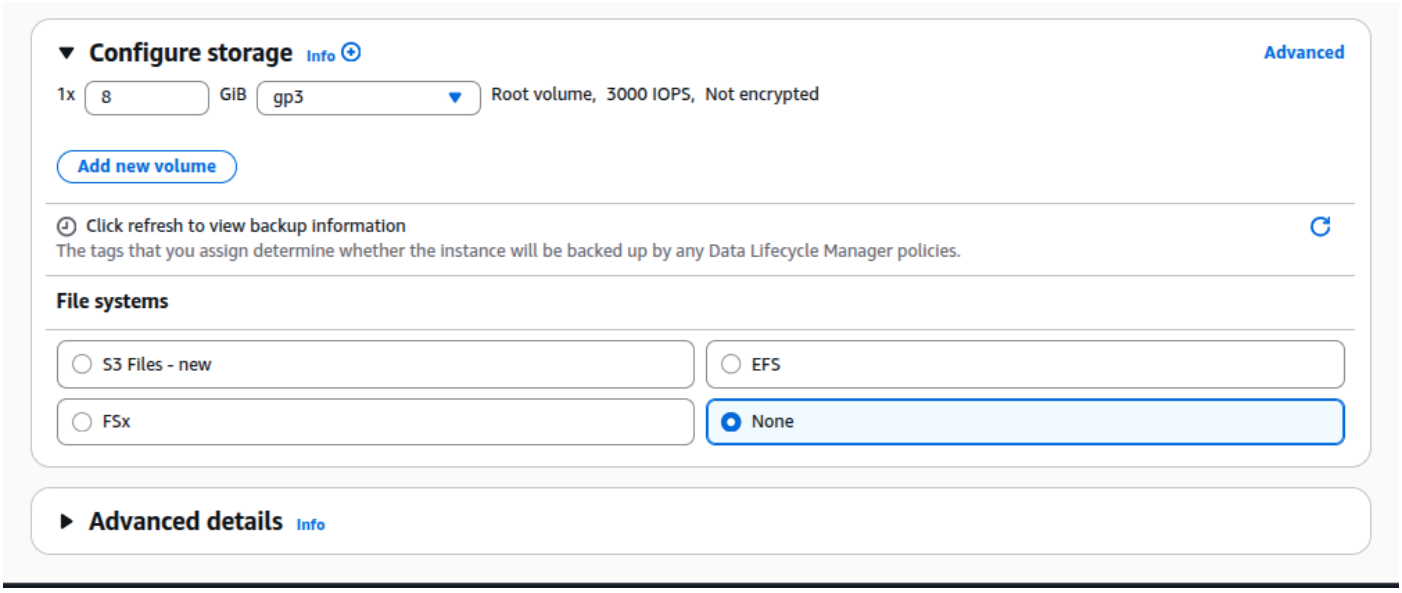
- Choose Amazon predefined AMI
- Choose instance type
- Create or choose Key pair



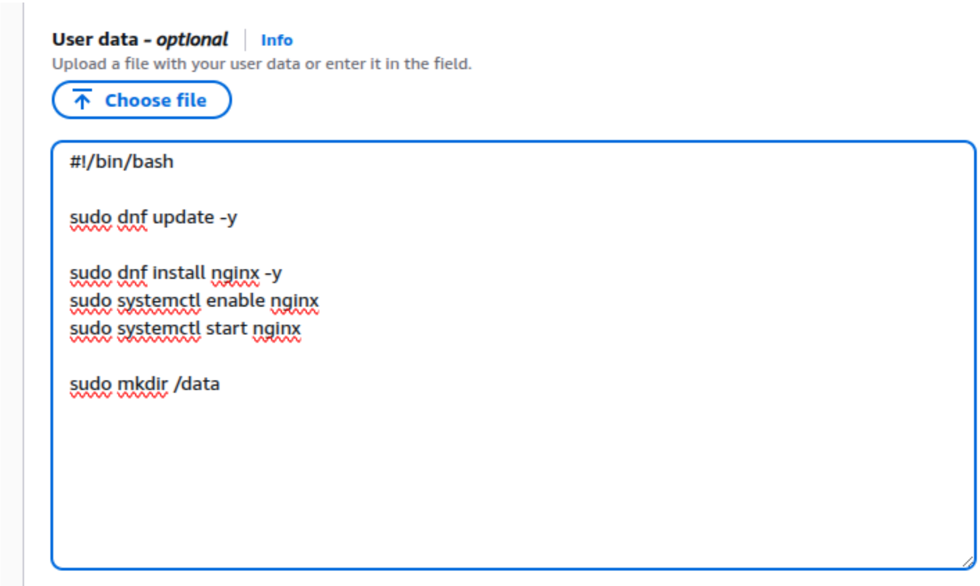
Network ,Storage and Firewall Settings

- Select VPC
- Enable Public IP
- Enable SSH (22) and HTTP (80) ports in Security group
- Choose root volume or additional storage (if required file system)

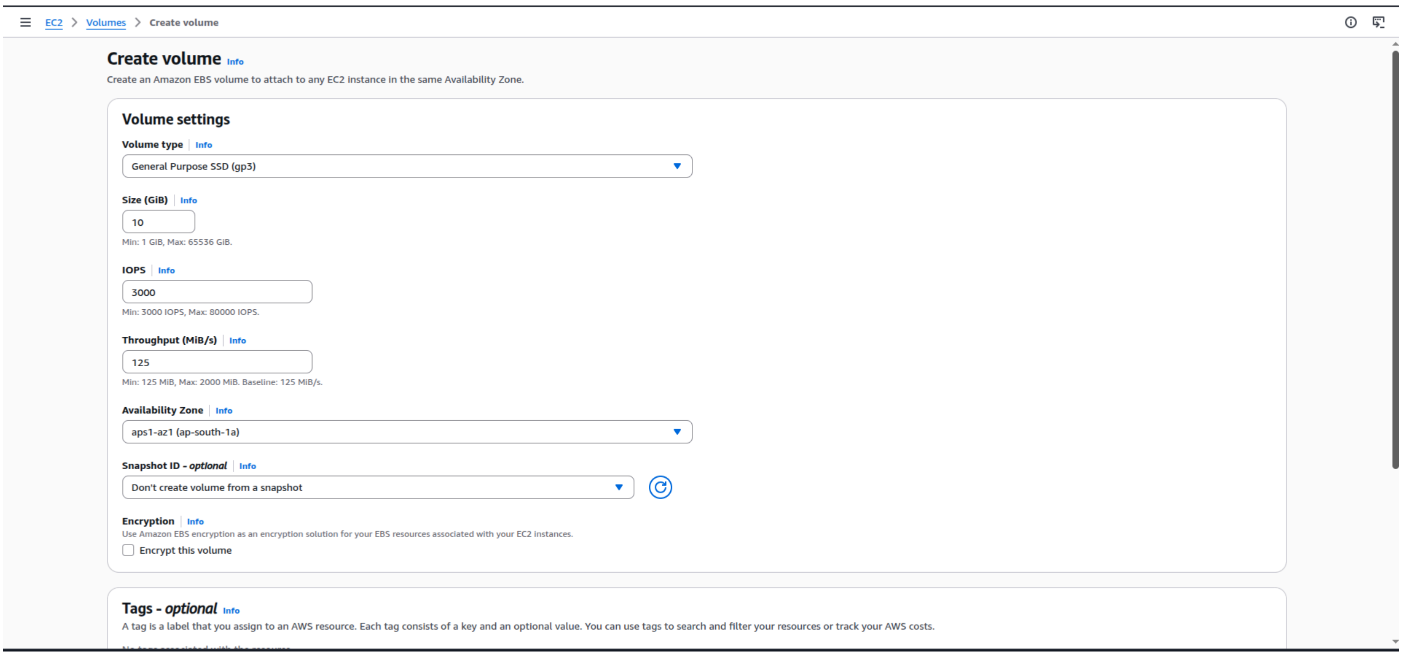




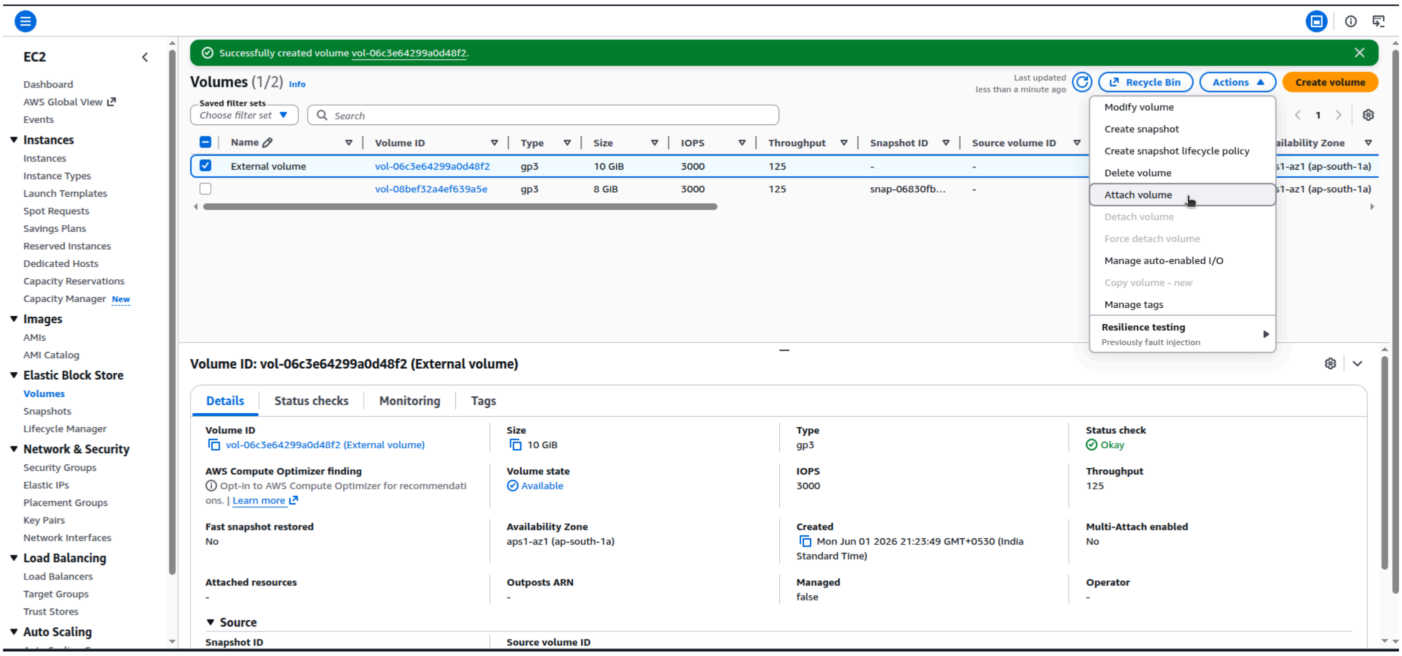
- Insert User data (initial script)

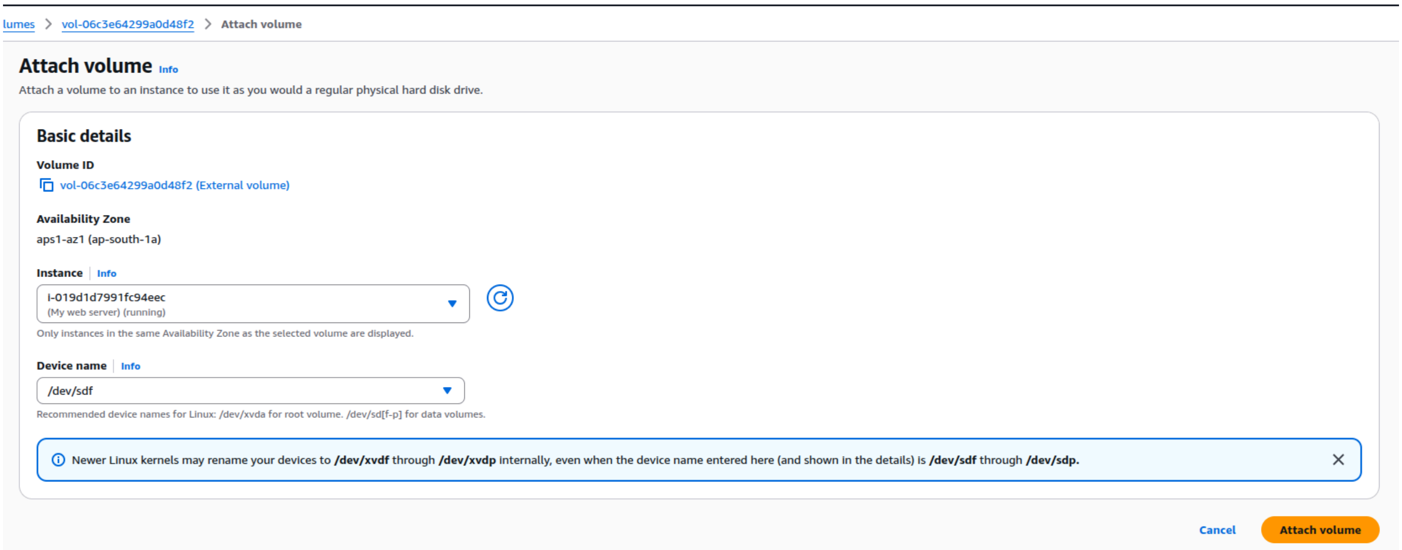


- Create additional volume as per requirement

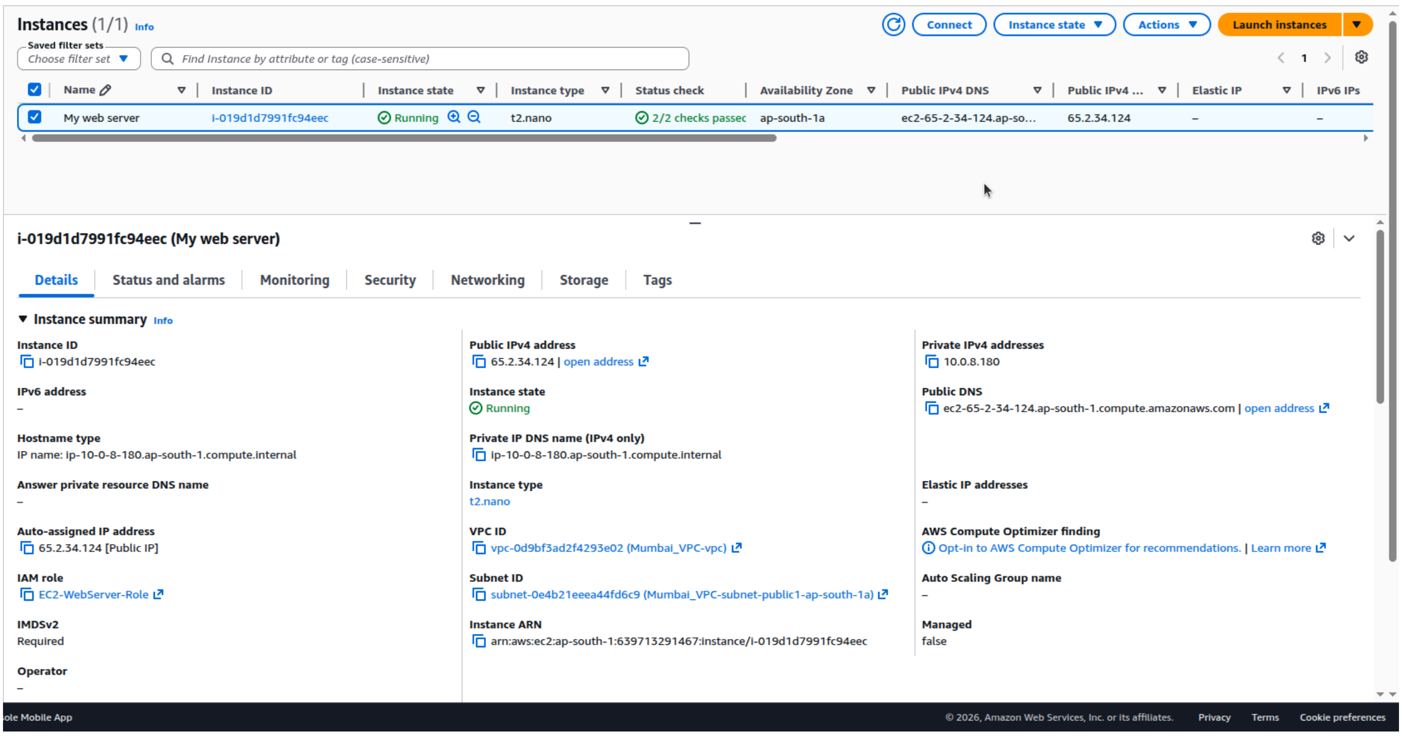


- Attach volume to EC2 instance



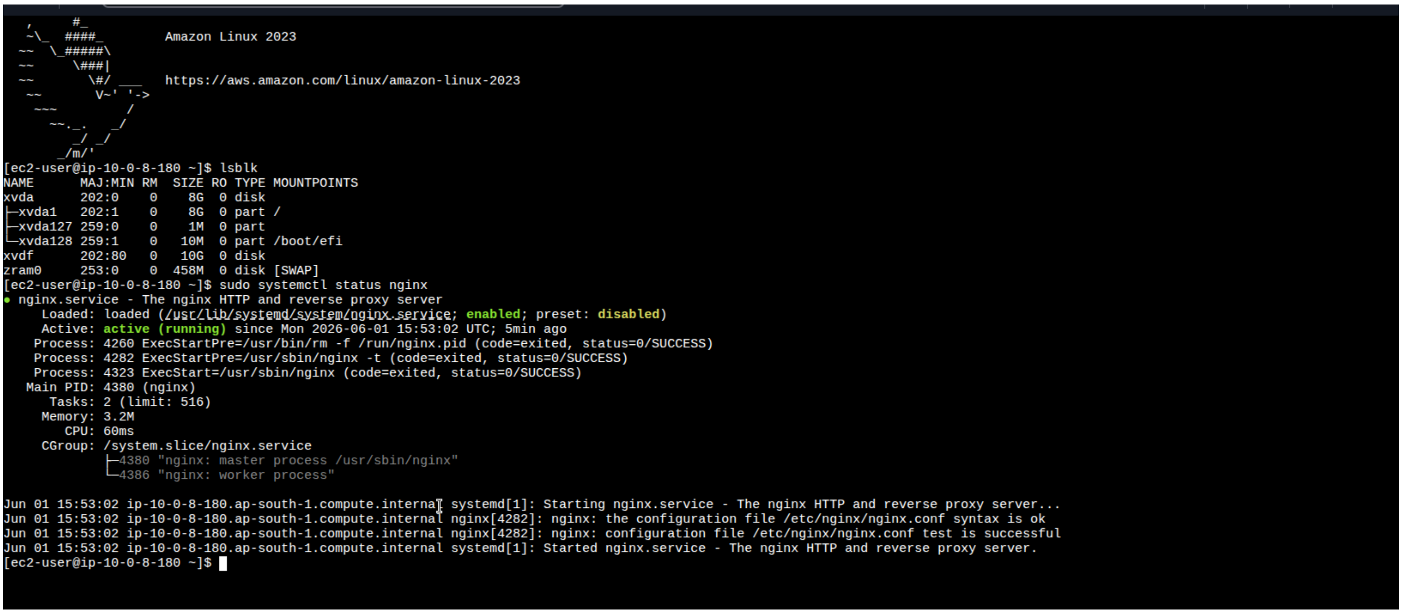


- Verify EC2 availability in the AWS console.



Access EC2 instance

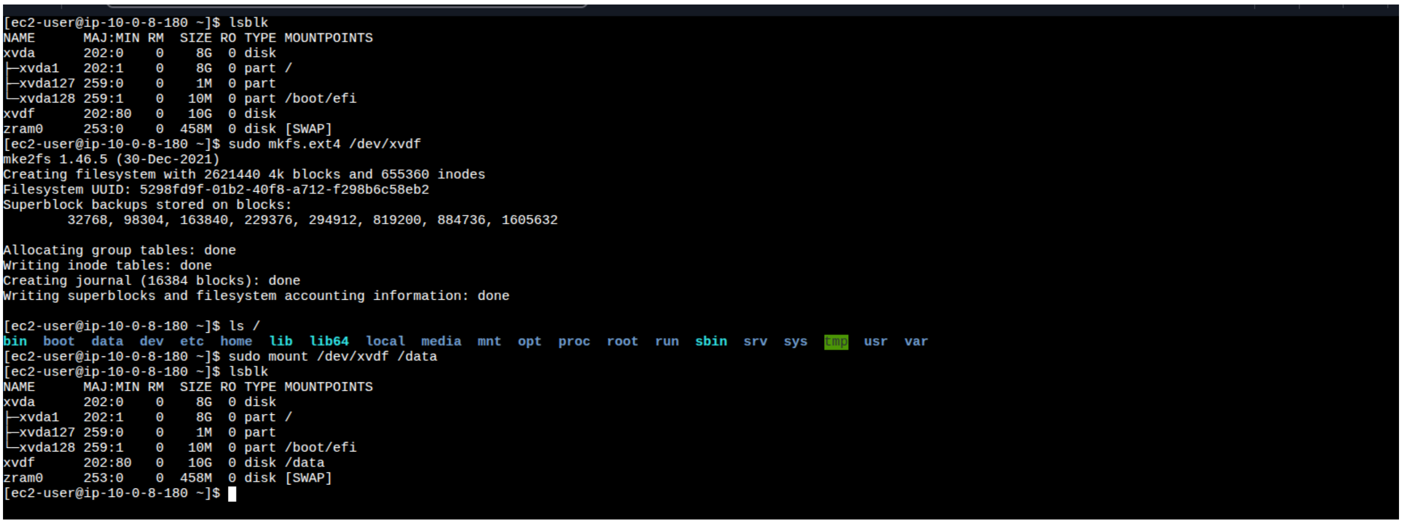
- Check storage
- Check Nginx service status



```
 ,     #_
 ~\_  ####_        Amazon Linux 2023
~~  \_#####\
~~     \###|
~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
 ~~       V~' '->
  ~~~         /
    ~~._.   _/
       _/ _/
     _/m/'
[ec2-user@ip-10-0-8-180 ~]$ lsblk
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
xvda      202:0    0    8G  0 disk
├─xvda1   202:1    0    8G  0 part /
├─xvda127 259:0    0    1M  0 part
└─xvda128 259:1    0   10M  0 part /boot/efi
xvdf      202:80   0   10G  0 disk
zram0     253:0    0  458M  0 disk [SWAP]
[ec2-user@ip-10-0-8-180 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-06-01 15:53:02 UTC; 5min ago
    Process: 4260 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Process: 4282 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
    Process: 4323 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 4380 (nginx)
      Tasks: 2 (limit: 516)
     Memory: 3.2M
        CPU: 60ms
     CGroup: /system.slice/nginx.service
             ├─4380 "nginx: master process /usr/sbin/nginx"
             └─4386 "nginx: worker process"

Jun 01 15:53:02 ip-10-0-8-180.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Jun 01 15:53:02 ip-10-0-8-180.ap-south-1.compute.internal nginx[4282]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jun 01 15:53:02 ip-10-0-8-180.ap-south-1.compute.internal nginx[4282]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jun 01 15:53:02 ip-10-0-8-180.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-10-0-8-180 ~]$
```

- Create file system and mount external storage using below commands
  - lsblk : show available storage
  - mkfs.ext4 /dev/xvdf : create file system of additional storage
  - mount /dev/xvdf /data : Mount created file system in over local folder
  - Sudo vi /etc/fstab : Edit file system file and add external storage UUID, type, etc..
    - UUID=”” /mount_location ext4 defaults,nofail 0 2
  - Sudo mount -a



```
[ec2-user@ip-10-0-8-180 ~]$ lsblk
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
xvda      202:0    0    8G  0 disk
├─xvda1   202:1    0    8G  0 part /
├─xvda127 259:0    0    1M  0 part
└─xvda128 259:1    0   10M  0 part /boot/efi
xvdf      202:80   0   10G  0 disk
zram0     253:0    0  458M  0 disk [SWAP]
[ec2-user@ip-10-0-8-180 ~]$ sudo mkfs.ext4 /dev/xvdf
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: 5298fd9f-01b2-40f8-a712-f298b6c58eb2
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

[ec2-user@ip-10-0-8-180 ~]$ ls /
bin  boot  data  dev  etc  home  lib  lib64  local  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
[ec2-user@ip-10-0-8-180 ~]$ sudo mount /dev/xvdf /data
[ec2-user@ip-10-0-8-180 ~]$ lsblk
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
xvda      202:0    0    8G  0 disk
├─xvda1   202:1    0    8G  0 part /
├─xvda127 259:0    0    1M  0 part
└─xvda128 259:1    0   10M  0 part /boot/efi
xvdf      202:80   0   10G  0 disk /data
zram0     253:0    0  458M  0 disk [SWAP]
[ec2-user@ip-10-0-8-180 ~]$
```

Phase 3

- Create website folder in external mounted storage

```
[ec2-user@ip-10-0-8-180 www]$ pwd
/data/html/www
[ec2-user@ip-10-0-8-180 www]$
```

- Create index.html file inside this folder

```
            background-color: #e68a00;
            cursor: pointer;
        }

        footer {
            background-color: var(--primary-color);
            color: #a0aec0;
            text-align: center;
            padding: 1.5rem;
            font-size: 0.9rem;
            margin-top: auto;
        }
    </style>
</head>
<body>

    <header>
        <h1>AWS Cloud Practice Deployment</h1>
        <p>Testing High Availabilty Website with Load Balancer with Auto Scaling Group Infrastructure</p>
        <div class="badge">HTTP Active</div>
    </header>

    <main>
        <section class="card">
            <h2>Success! Your Web Server is Live</h2>
            <p>If you can see this page over a secure URL (starting with <strong>https://</strong>), it means your SSL/TLS certificates, security groups, and web server configurations are working properly.</p>

            <ul class="status-list">
                <li><strong>EC2 Instance:</strong> Create Web server.</li>
                <li><strong>IAM Role:</strong> Use for limited access</li>
                <li><strong>Security Groups:</strong> Configured to allow HTTP (80)</li>
                <li><strong>ELB:</strong> Use external storage for store website data (prevent data loss).</li>
                <li><strong>ALB:</strong> Application Load Balancer</li>
                <li><strong>Web Server:</strong> Document root successfully updated.</li>
            </ul>

            <a href="#" class="btn" onclick="alert('Verification successful! Keep up the great work learning AWS.')">Verify Connection</a>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 AWS Practice Lab | Built with HTML & CSS</p>
    </footer>

</body>
</html>
                                                                                161,7          Bot
```

i-019d1d7991fc94eec (My web server)

PublicIPs: 65.2.34.124 PrivateIPs: 10.0.8.180

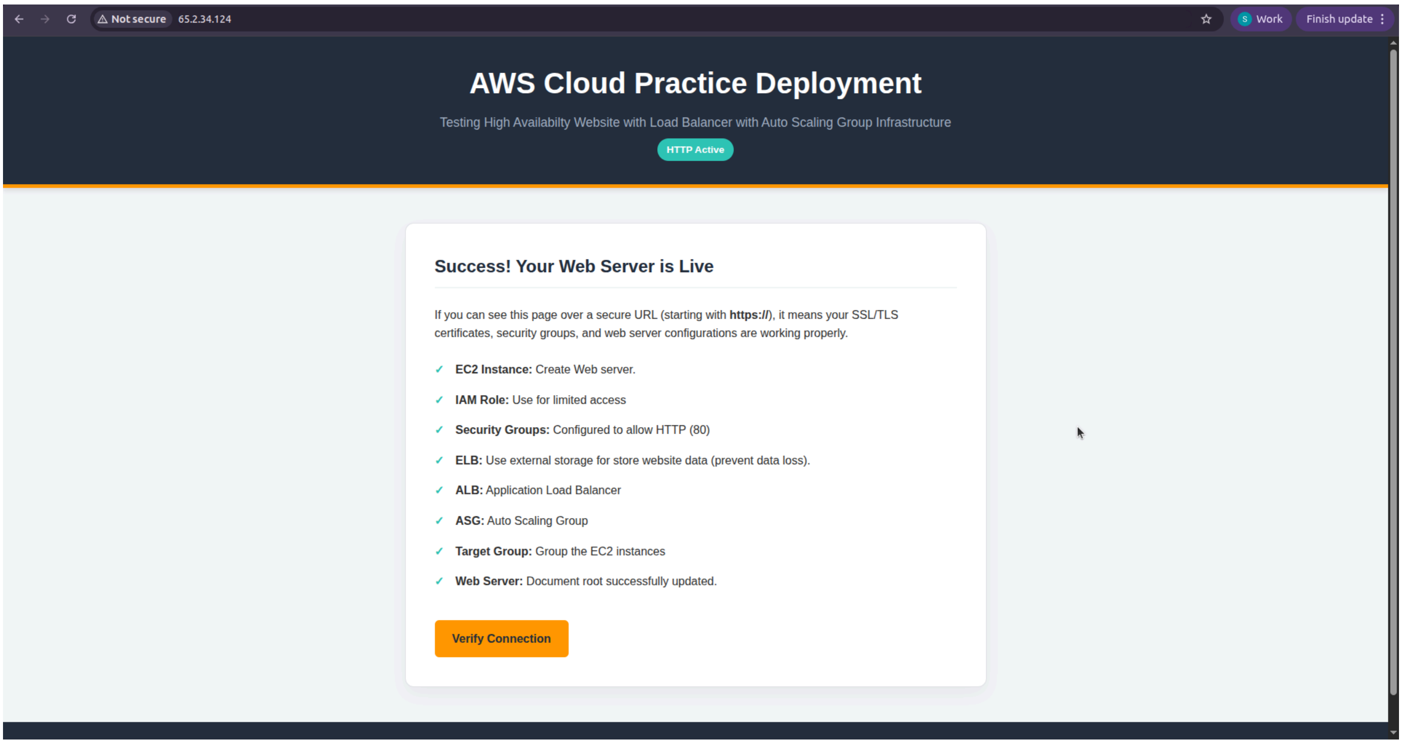
- Edit nginx config file
    - Edit root folder location: replace default location to our website folder

```
    server {
        listen       80;
        listen       [::]:80;
        server_name  _;
        root         /data/html/www;

        # Load configuration files for the default server block.
```

- Restart nginx service after configuration changes

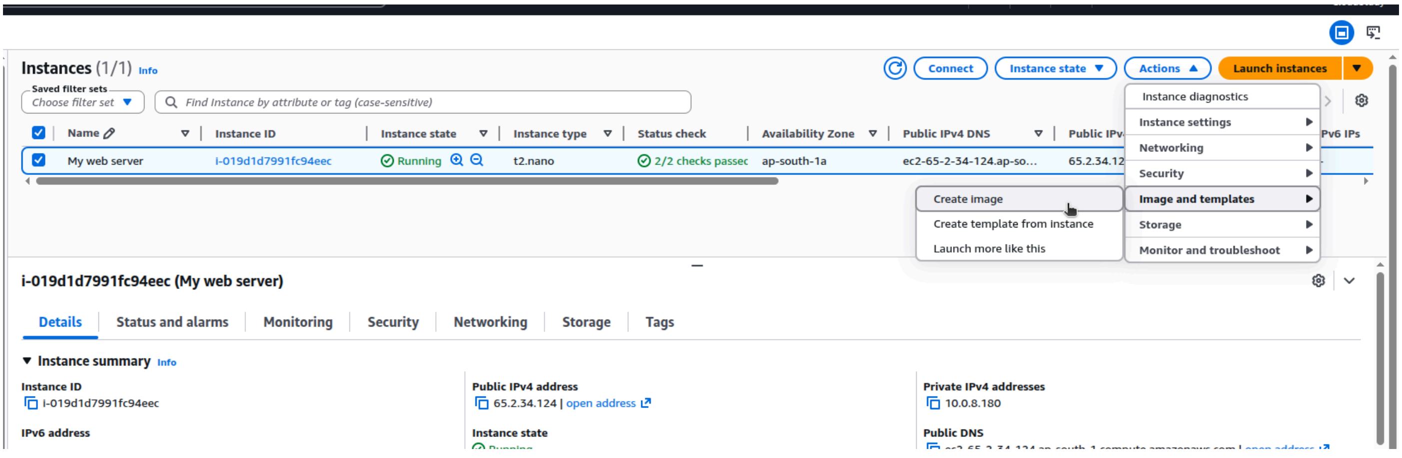
- **Website is live**

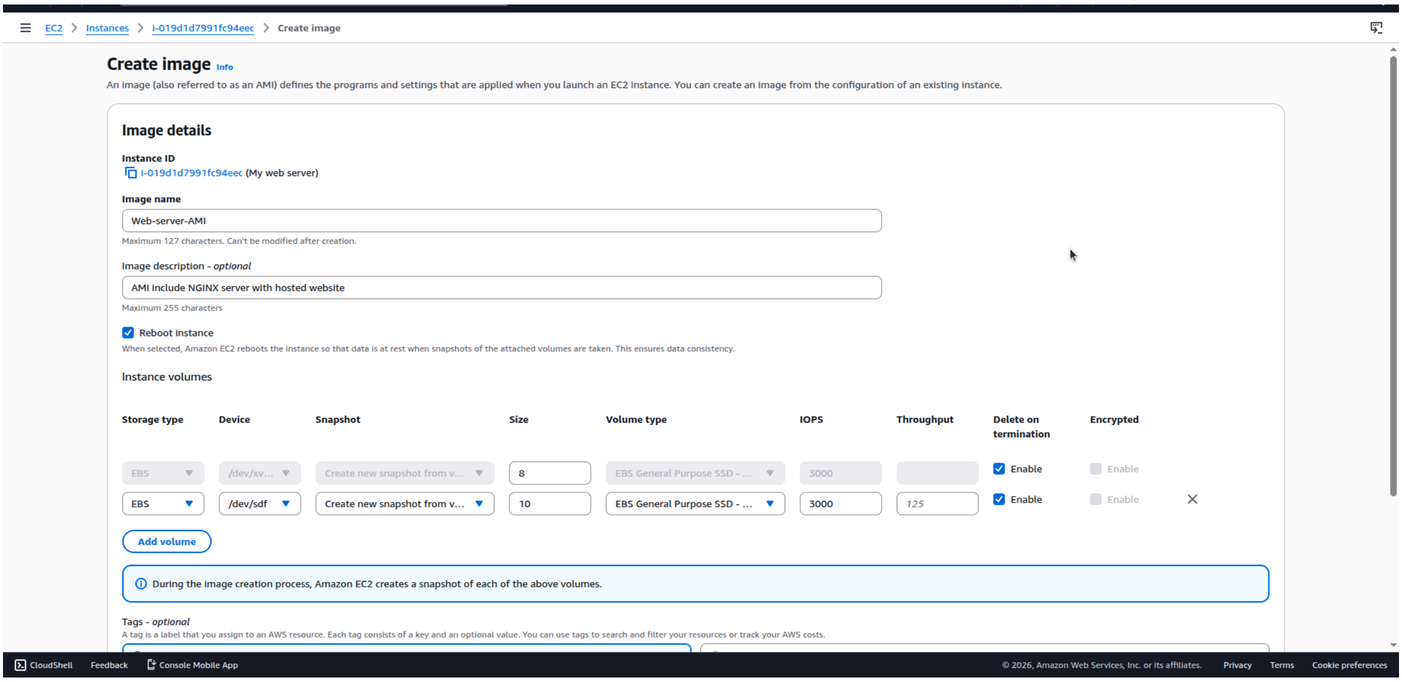


Phase 2

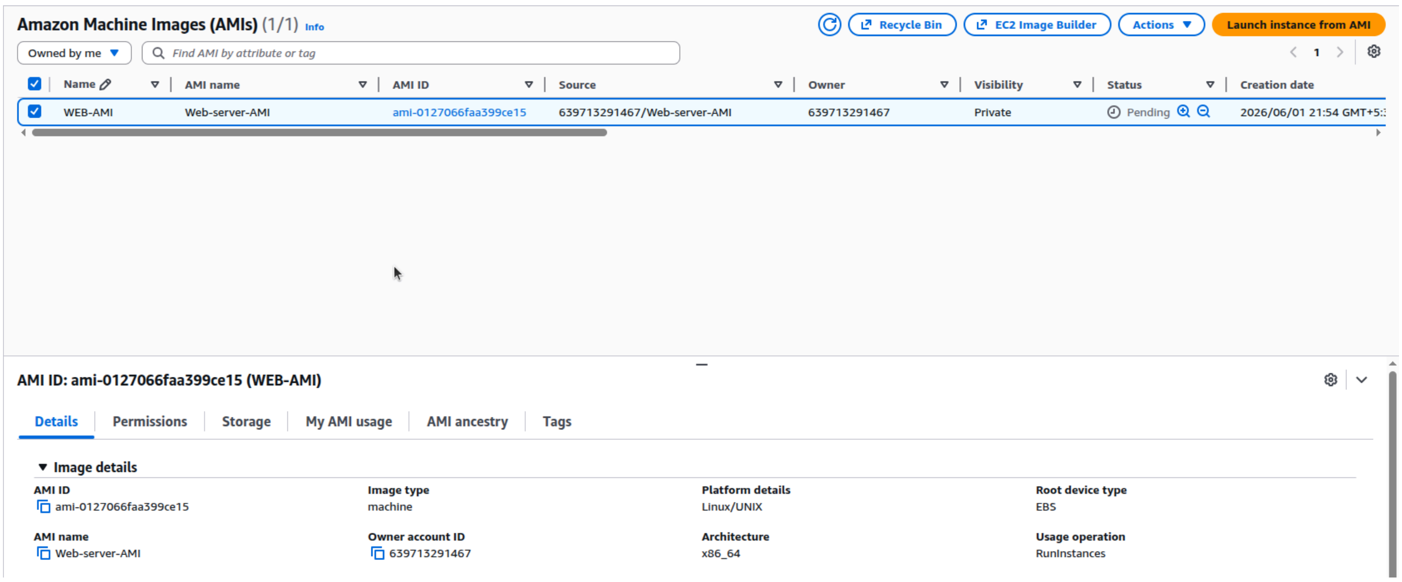
Create Amazon Machine Image (AMI - Golden Image)

- Select EC2 instance to create AMI
- Go to Action -> Image and template -> create image





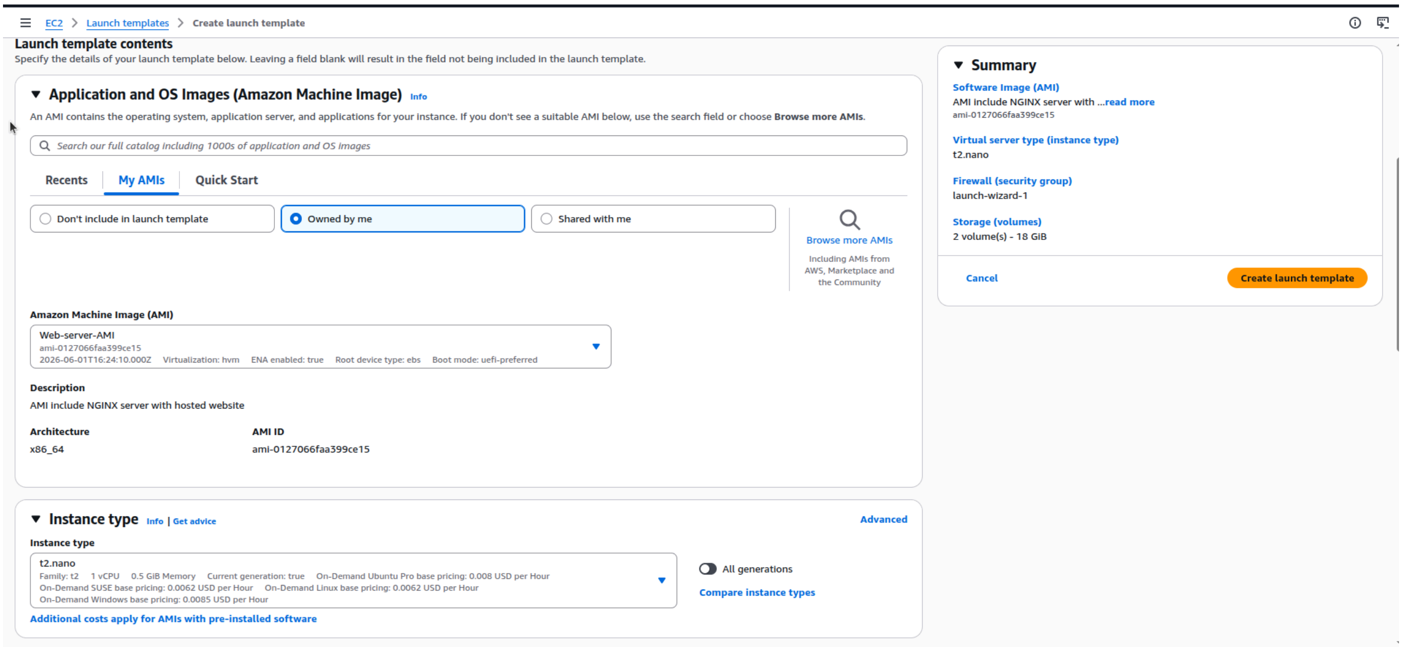
- Verify AMI availability in the AWS console.



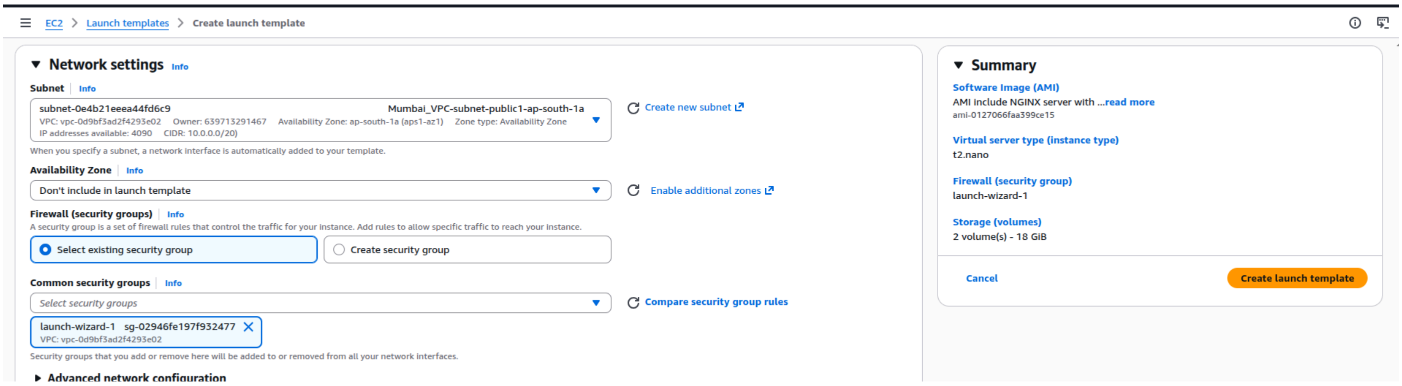
Phase 3

Create EC2 Launch Template

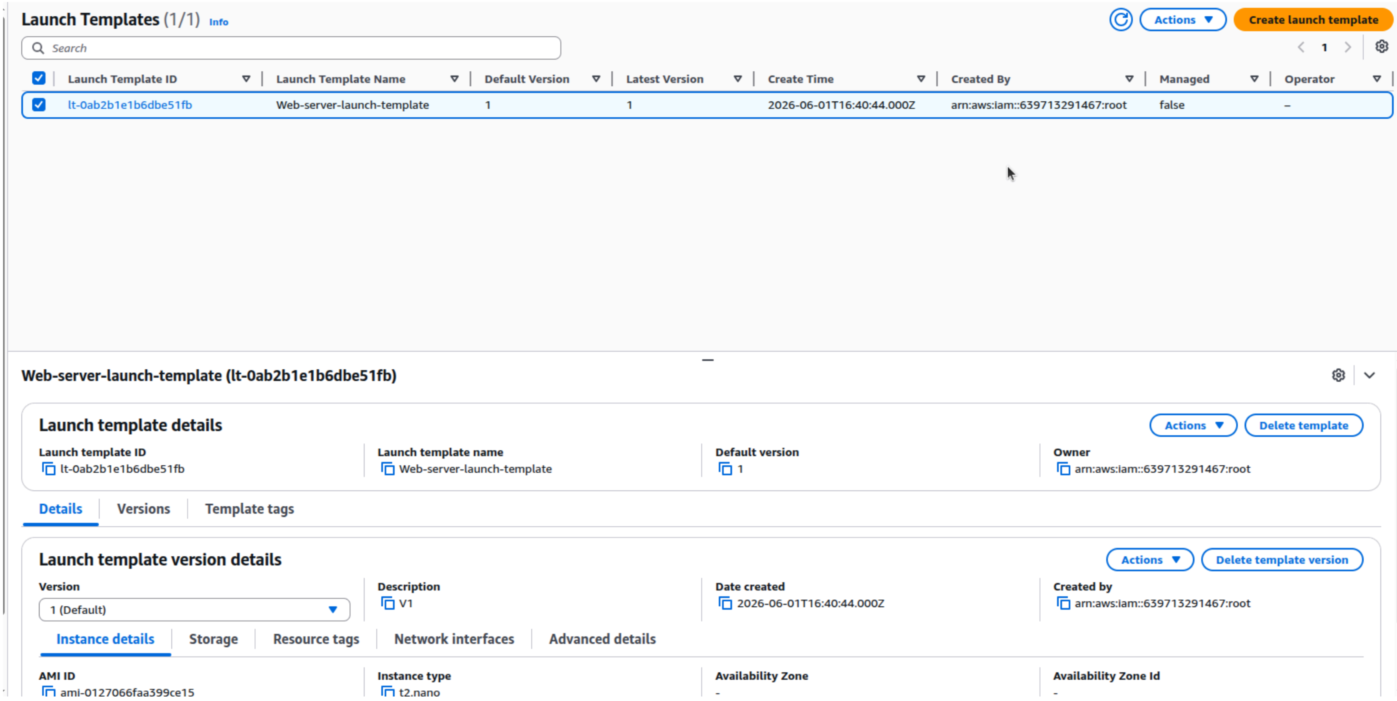
- Choose AMI from phase 2
- Choose instance type as per requirement



- Choose Network setting and Enable auto assign public IP option from advanced network configuration



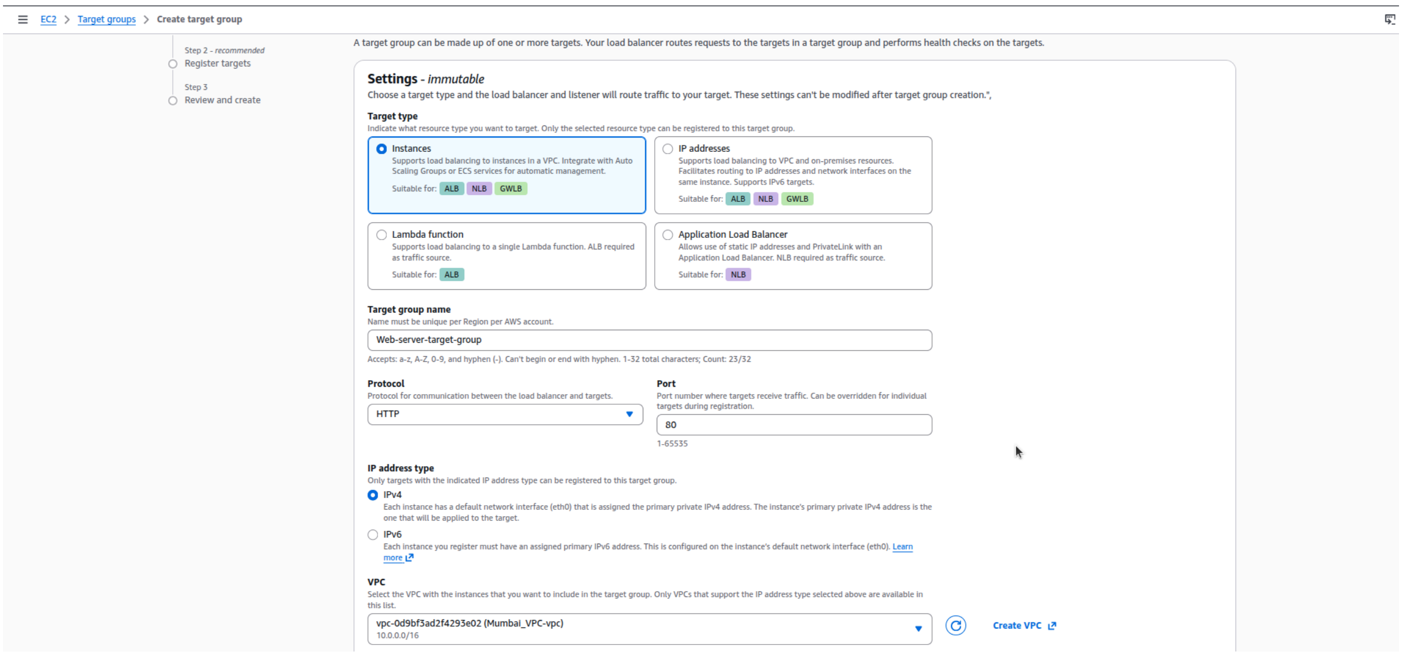
- Verify Launch template in AWS console



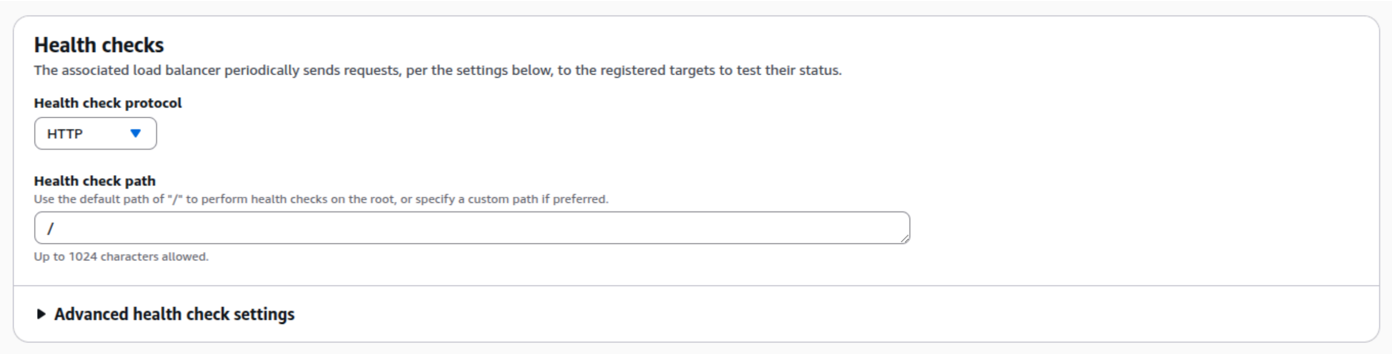
Phase 4

Create target group

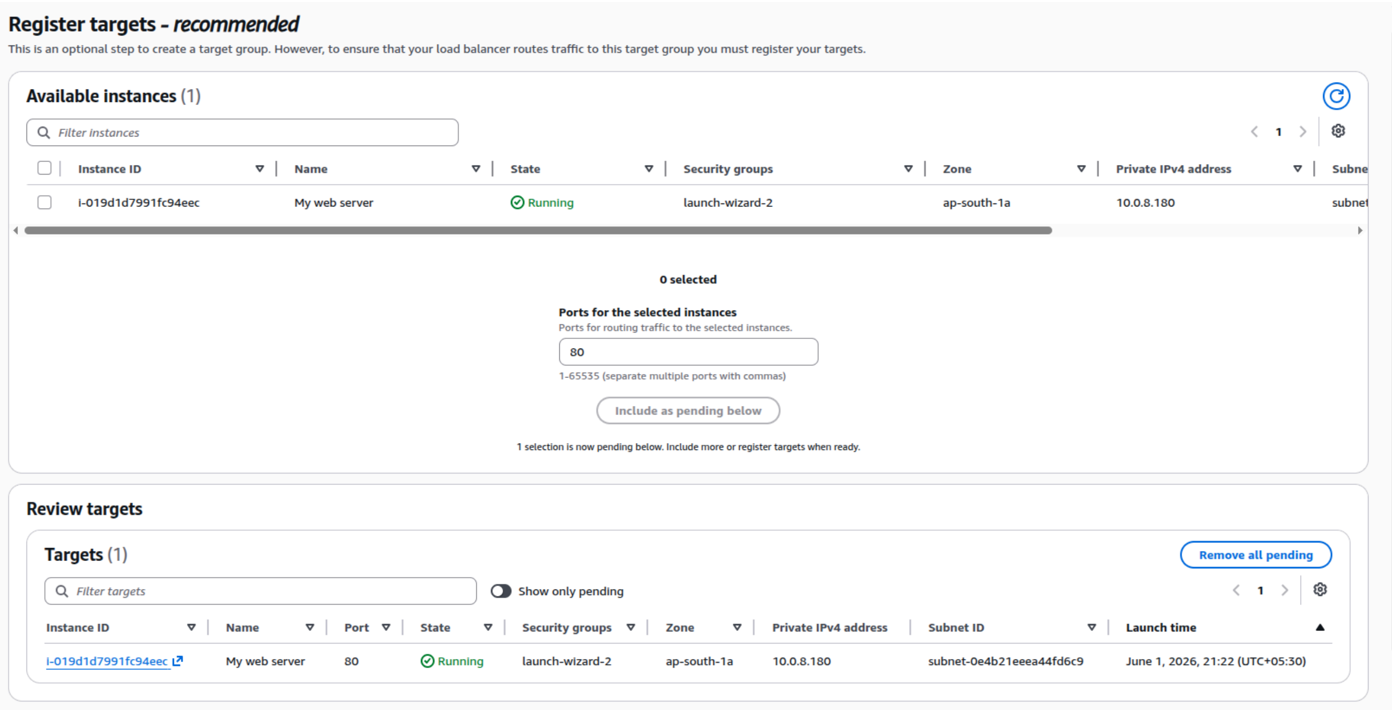
- Choose target type as per our requirement
- Choose required protocol
- Choose IP address type
- Select VPC



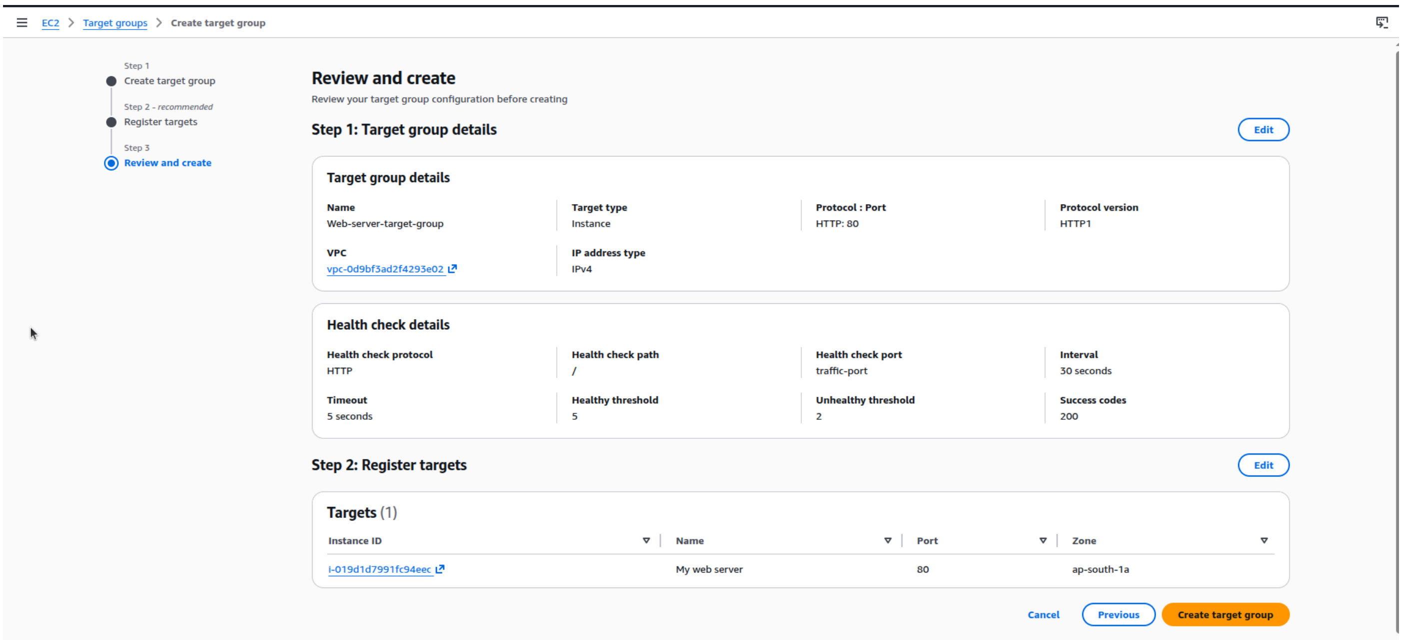
- Choose Health check Protocol
- Give the health check path (default root[/])
- Edit Advanced check as per requirement



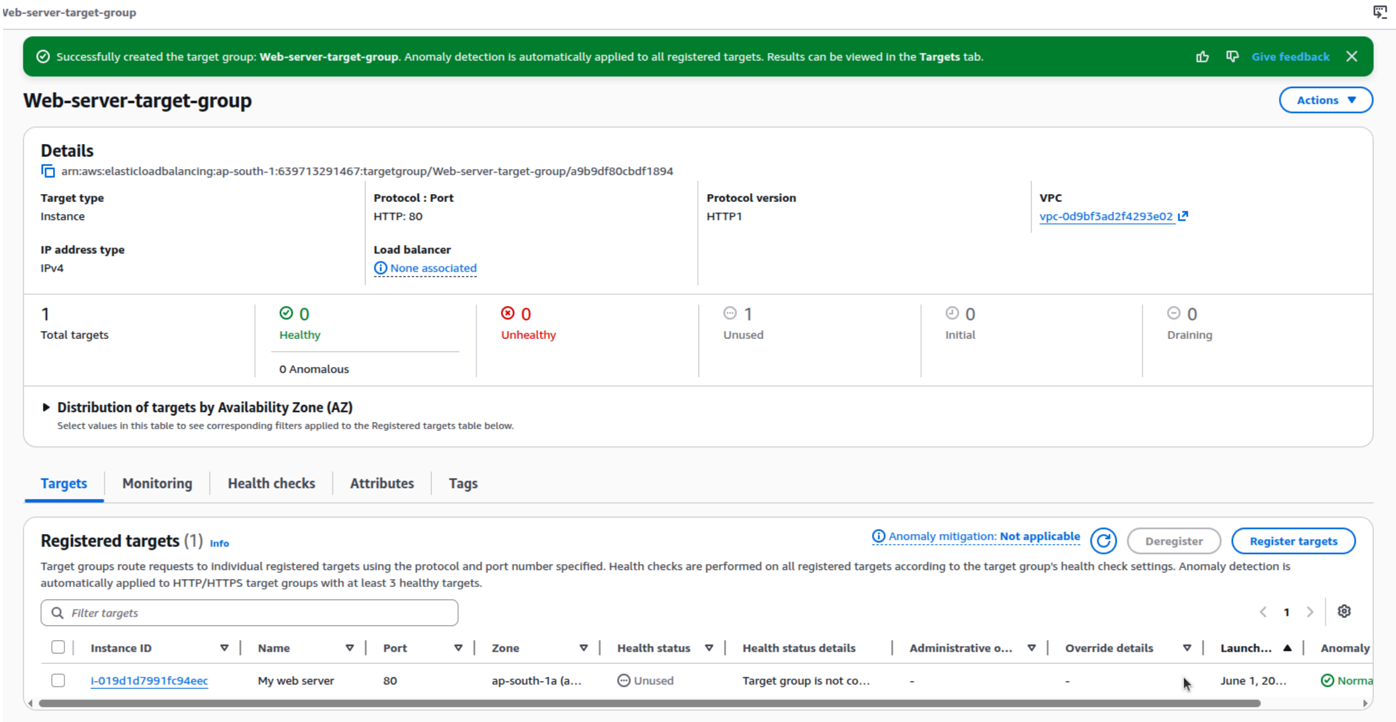
- Add EC2 instance in target group



- Target group review



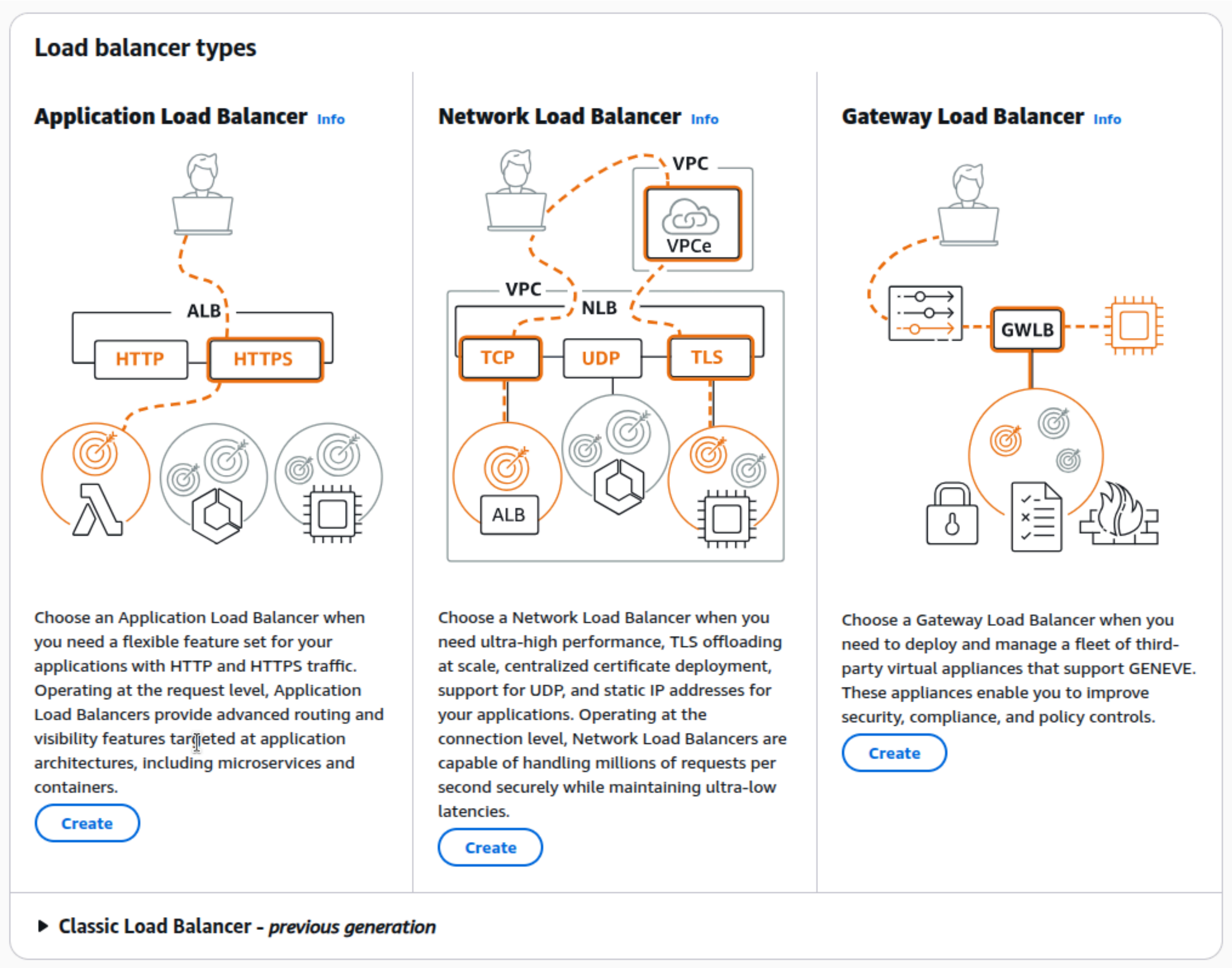
- Verify target group in AWS Console



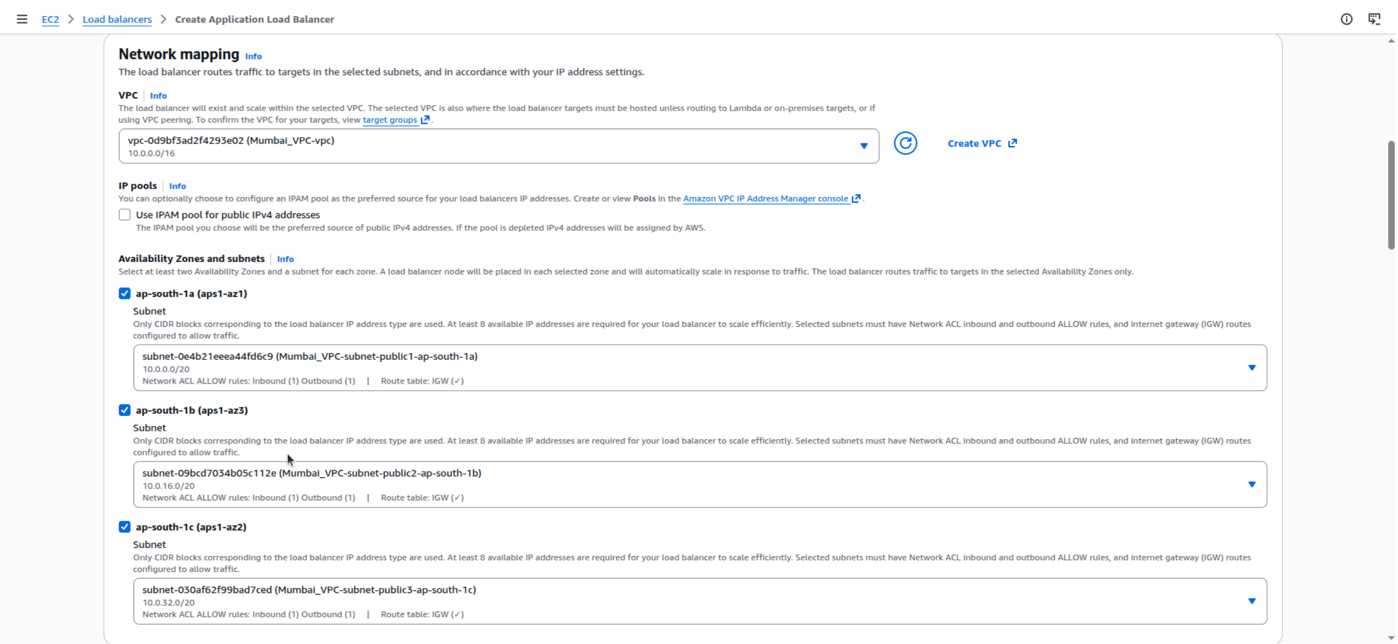
Phase 5

Create Load Balancer

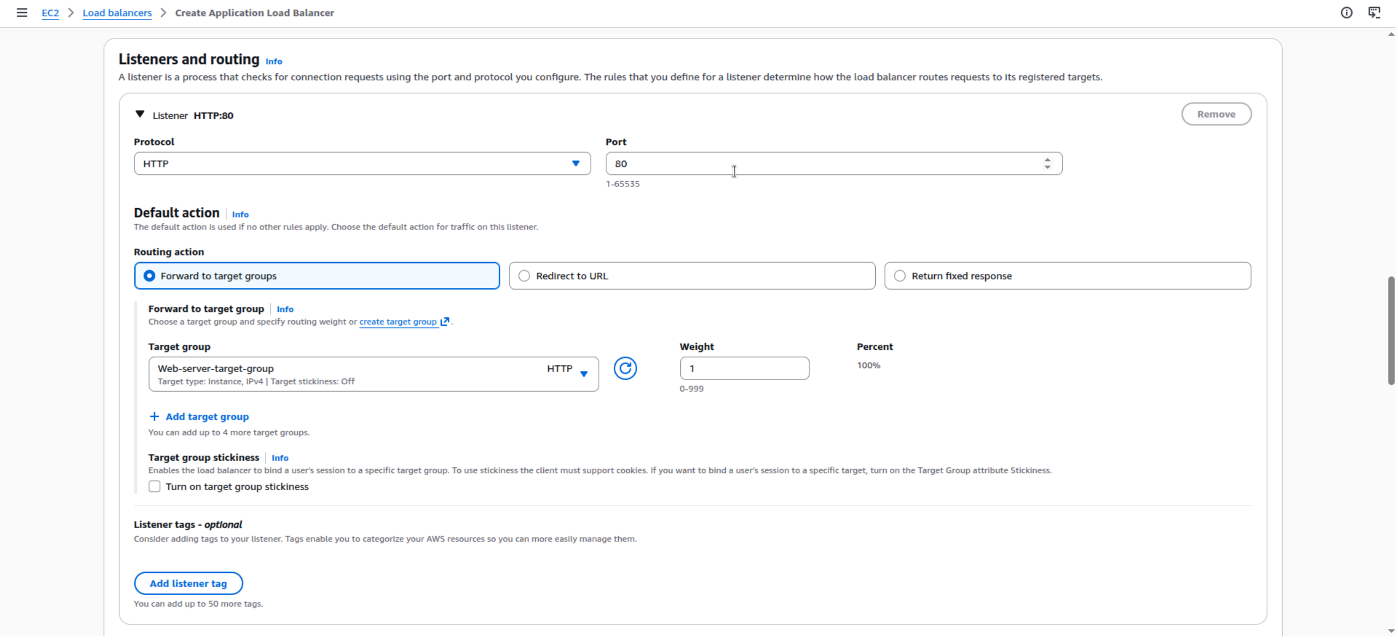
- Choose type of load balance (ALB)
- Choose scheme (internet or intranet facing)



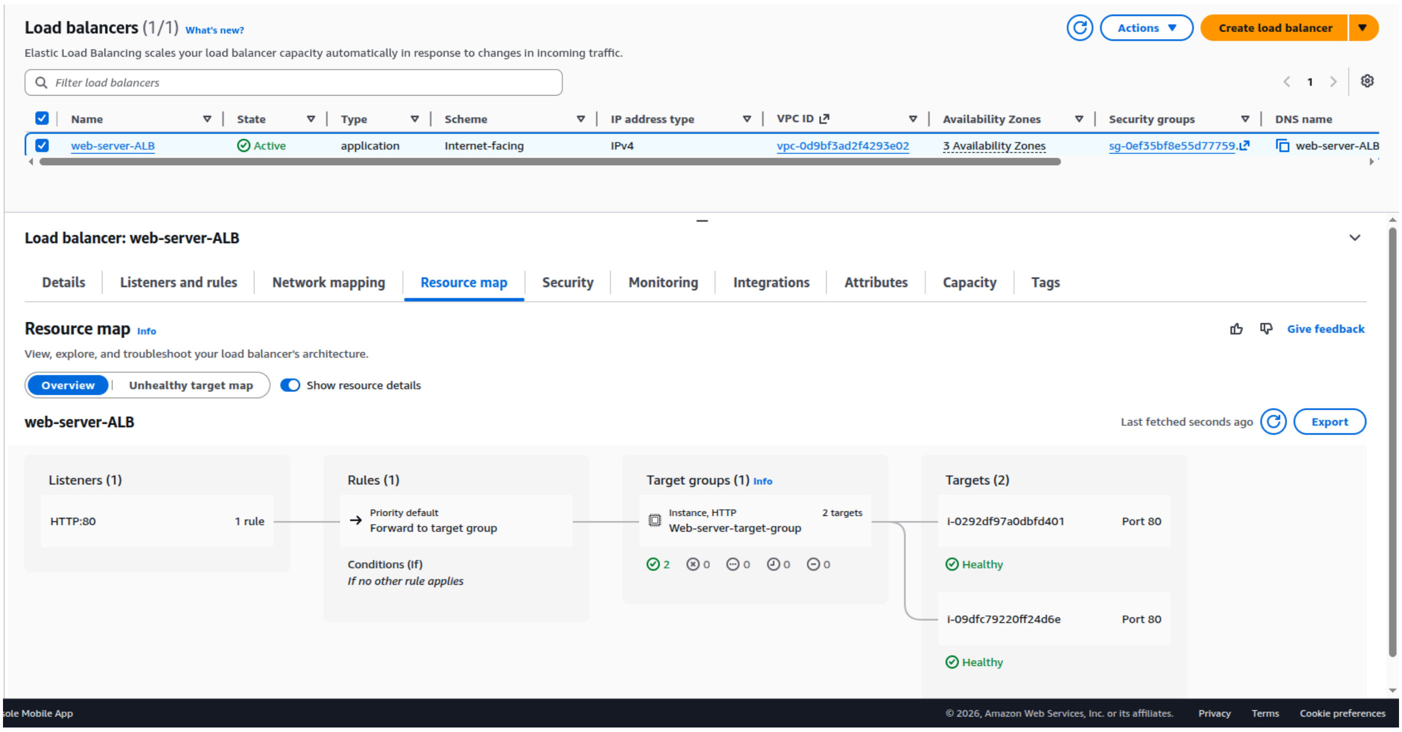
- ALB Network mapping and choose AZs



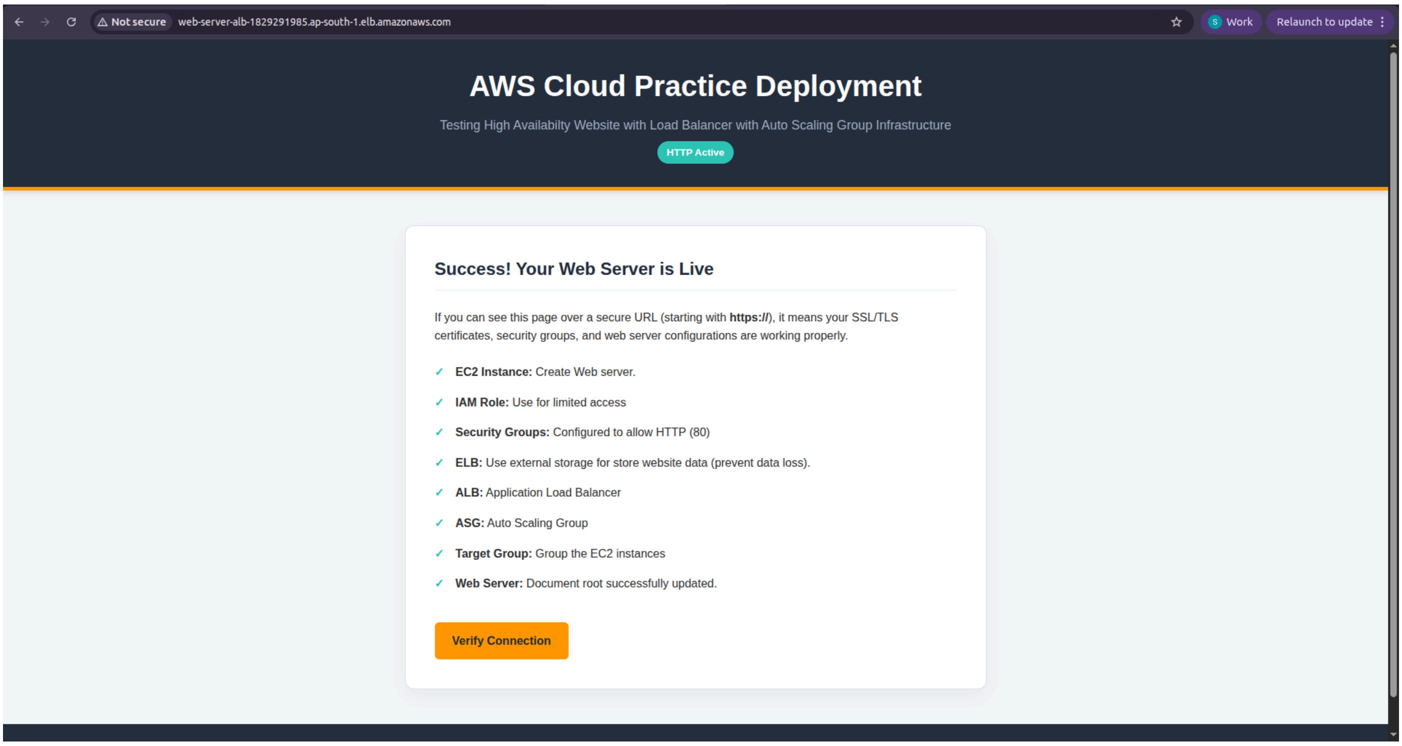
- Configure Listener and Routing
  - Choose listener port
  - Choose target group
  - Choose wight to move traffic



- Verify ALB in AWS console



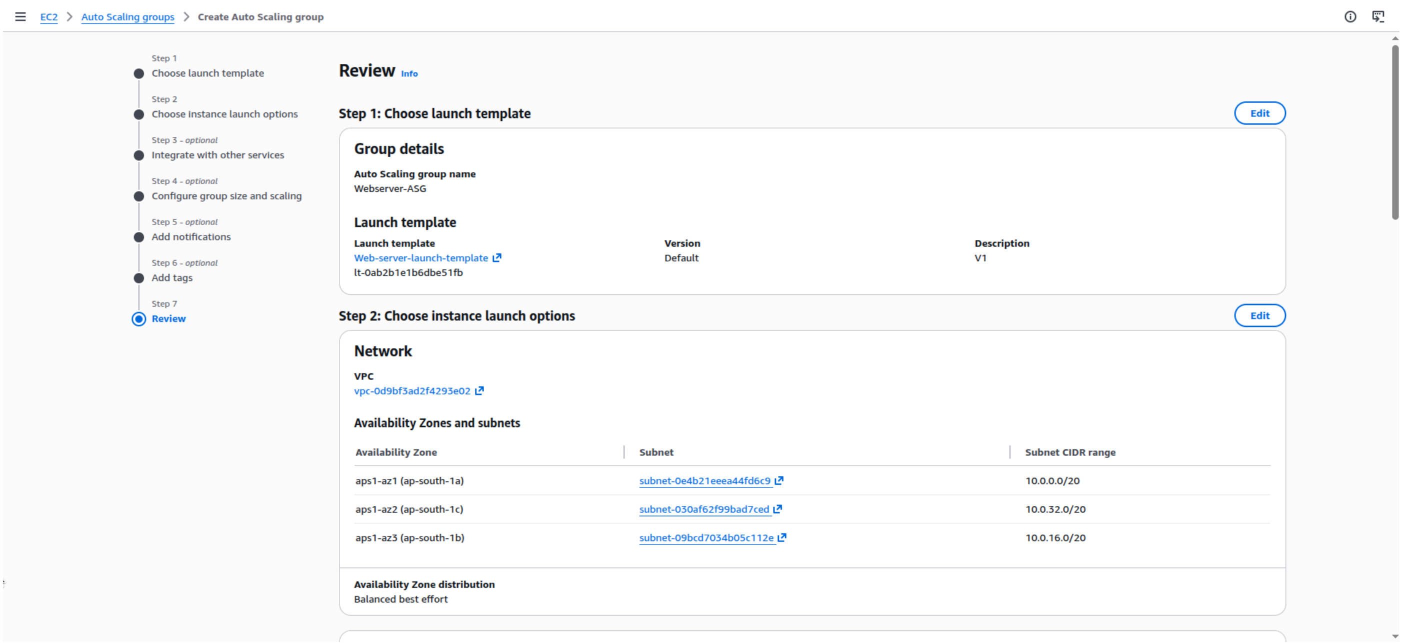
- Get confirmation website live or not (chose ALB DNS name and paste in chrome)

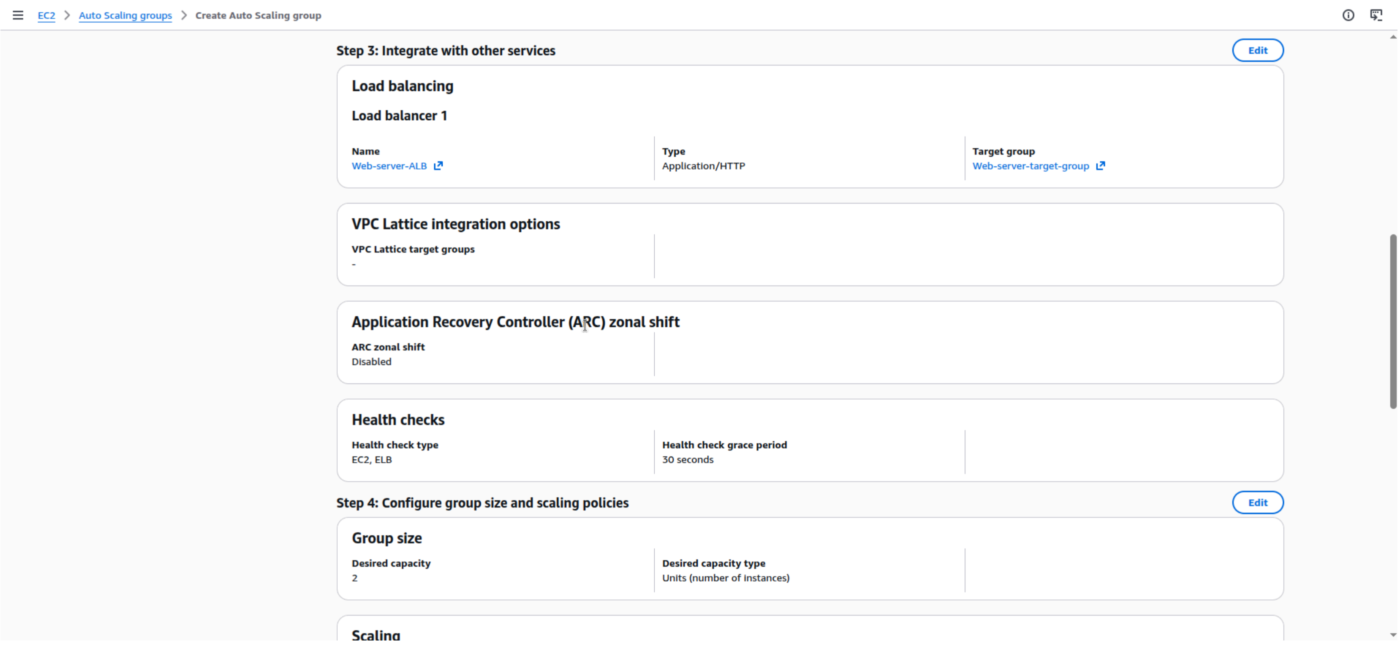


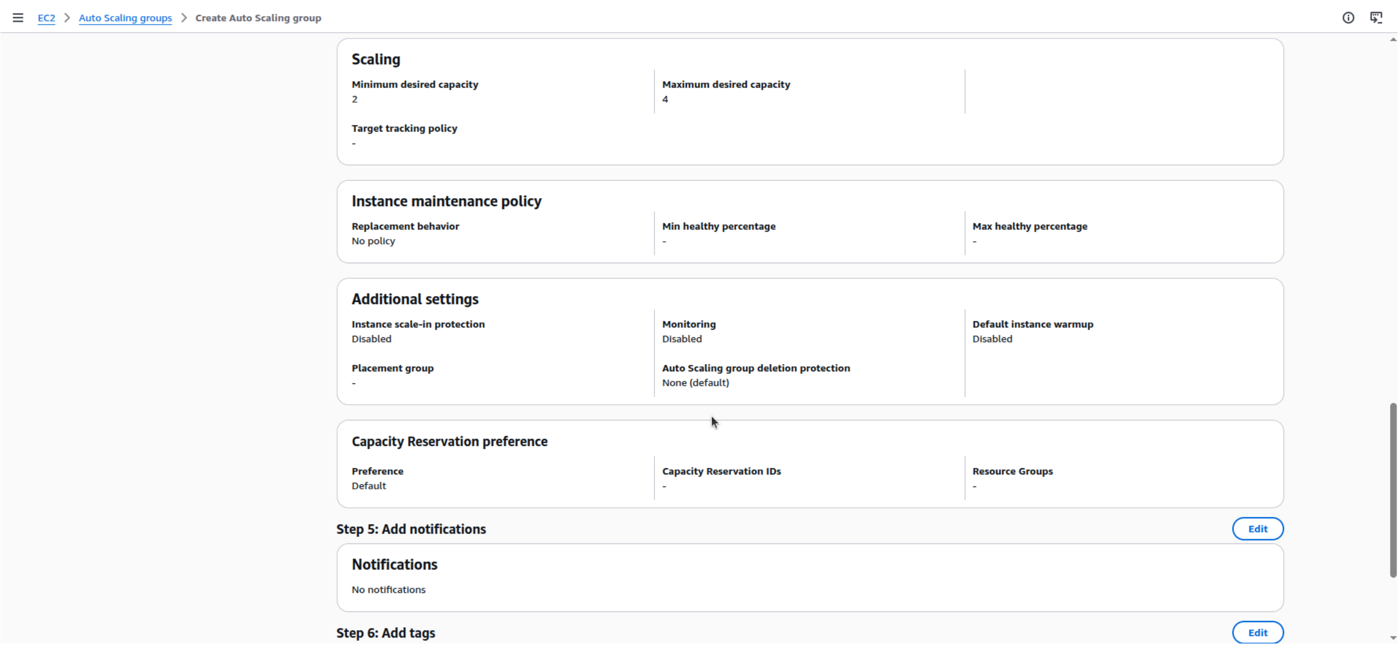
Phase 6

Create Auto Scaling Group

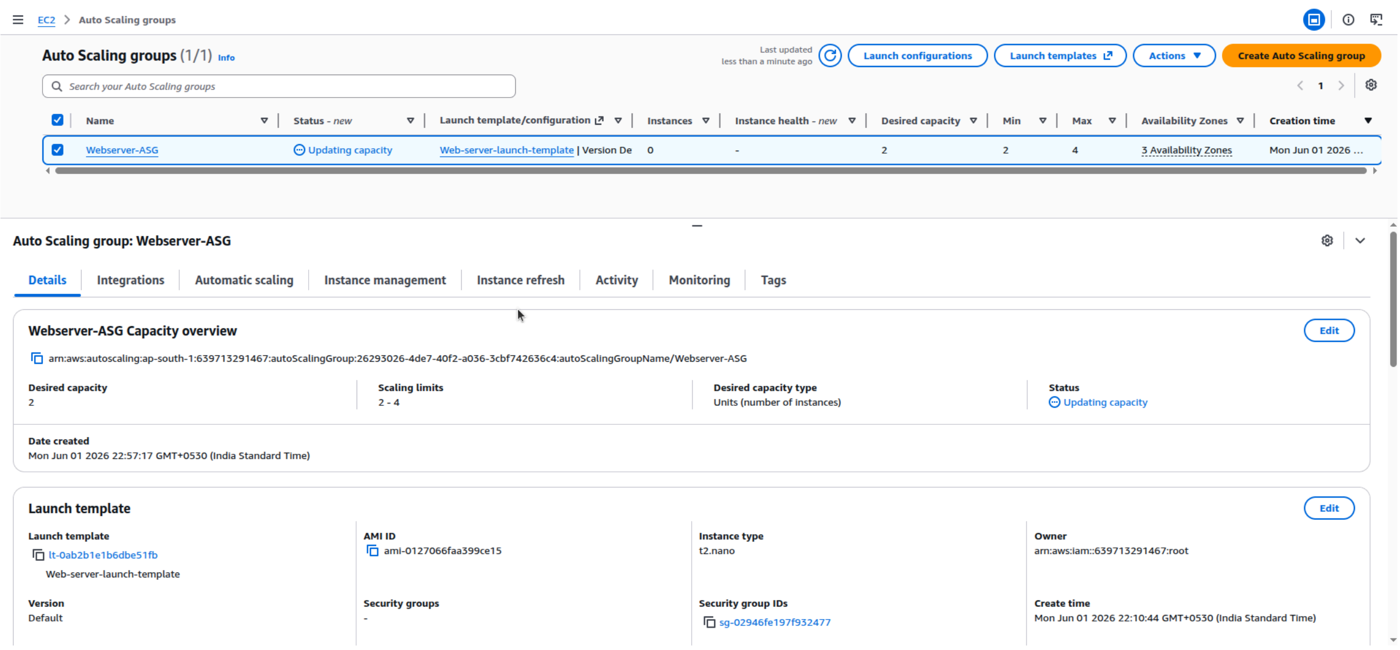
- Choose Launch template
- Choose Network setting and AZs
- Choose Load balancing
- Choose Load balancing target group
- Turn on ELB health check
- Choose health check period
- Choose group size (desire) and scaling size (min and max)



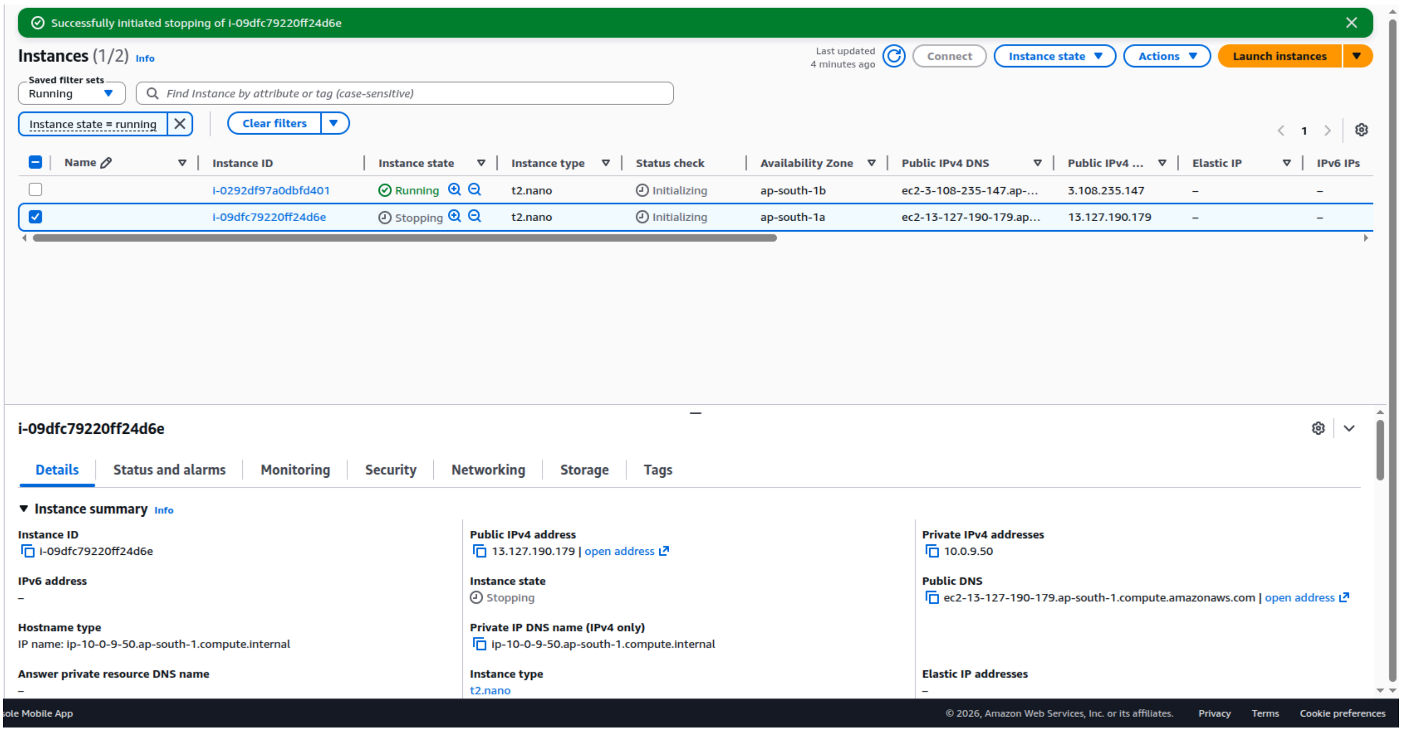




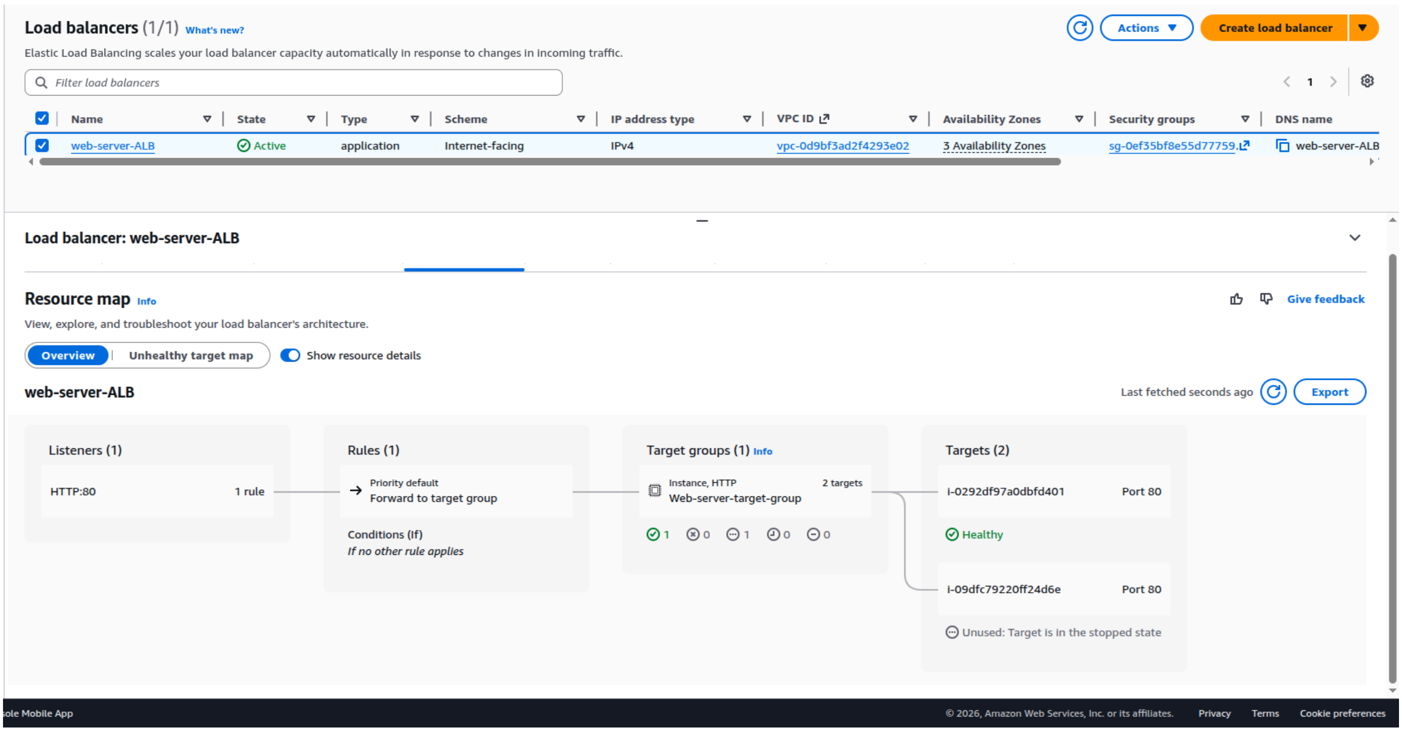
- Verify ASG in AWS console



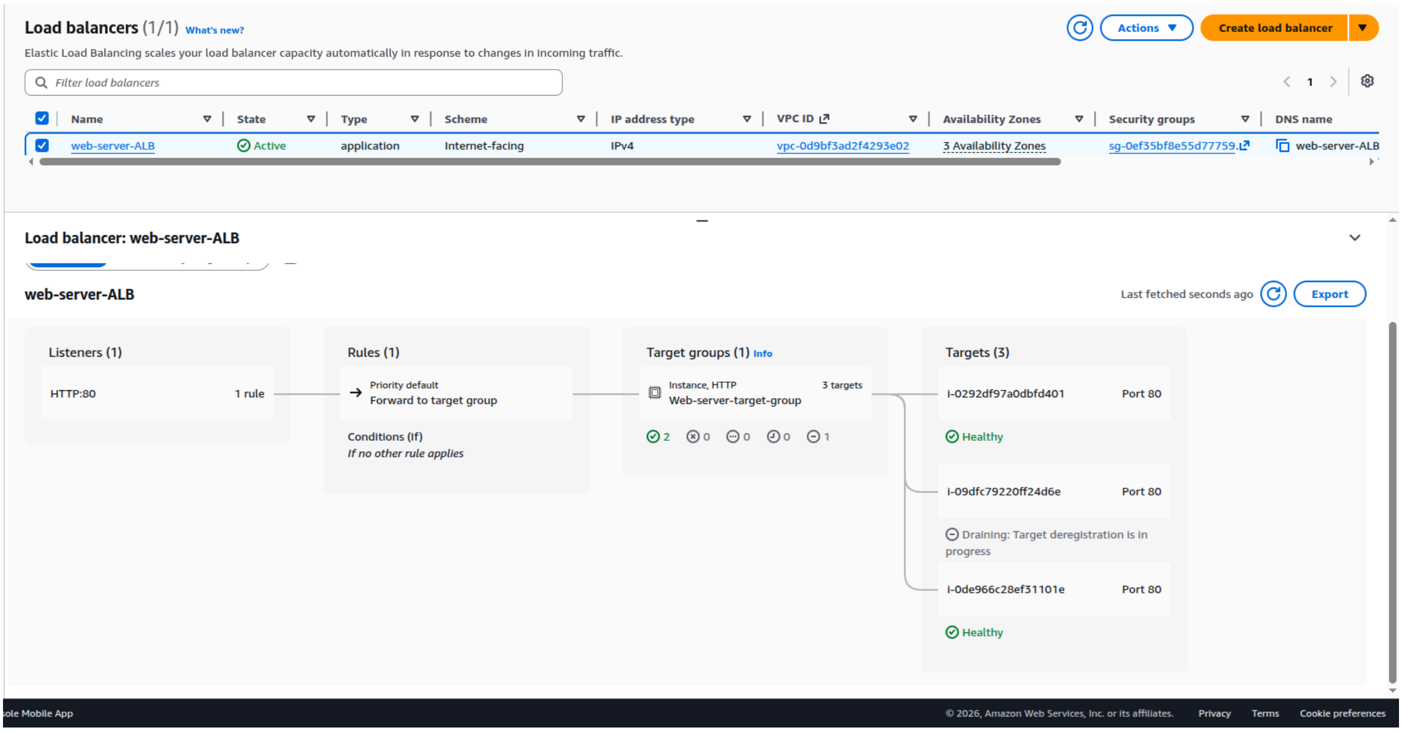
- Stop one EC2 to check fault tolerance and high availability



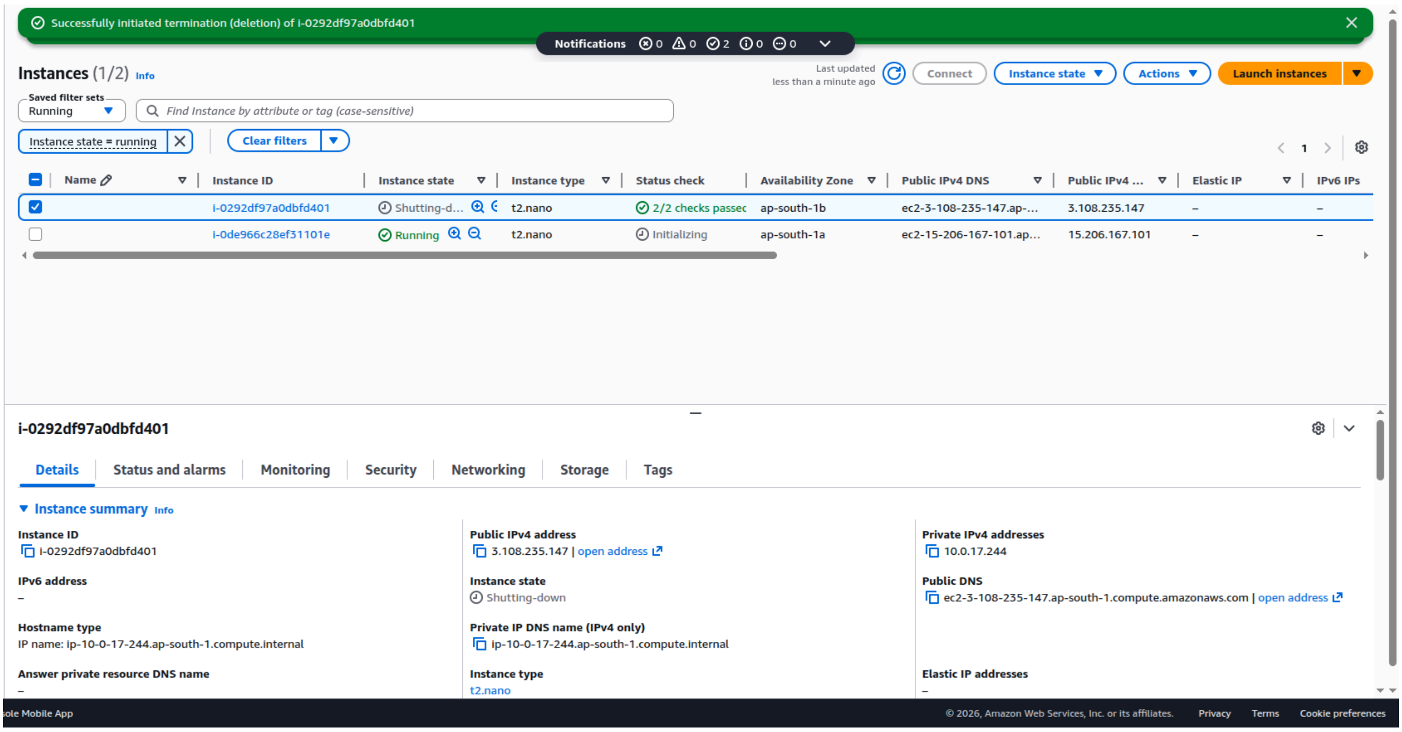
- In ALB go to unuse state



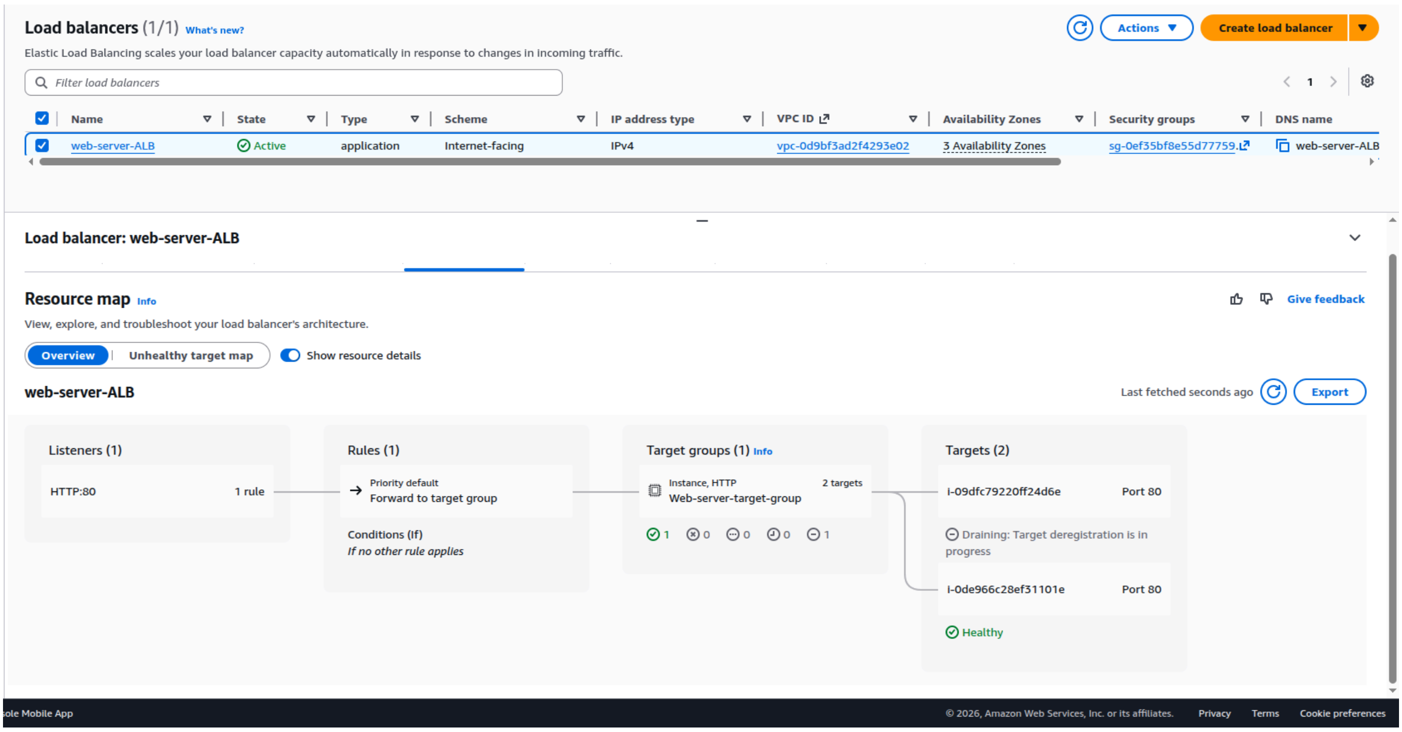
- ASG Scale up new instance after stopping one



- Terminate one instance



- ALB show only one live instance



- ASG auto created 1 more